AI-Powered Store Availability Dashboard

Prueba Técnica · Rappi Engineering 2026

TECH STACK
LIBRE

FORMATO
INDIVIDUAL

React 19 · Vite 6 · Recharts · Claude API · Anthropic

Juan Blanco

21 Abril 2026

La disponibilidad de tiendas es un dato crítico.

Cada cambio de estado online/offline de una tienda en la plataforma Rappi genera un registro en tiempo real. Estos eventos afectan directamente la experiencia del usuario y los ingresos operacionales.

> *"Queremos ver cómo piensan, no solo qué entregan."*
>
> — Rappi Engineering

CRITERIOS DE EVALUACIÓN

30%	Uso efectivo de AI
25%	Funcionalidad
20%	Creatividad y UX
15%	Presentación
10%	Calidad del código

Arquitectura de la Aplicación

CSV Dataset

↓

Anthropic API

↓

01

Dashboard de Visualización

- 5 KPI Cards con sparklines
- Gráfico de serie temporal (Recharts)
- Golden Signals panel (SRE)
- Anomaly Detection AI
- SLO Summary con error budget
- Carga de CSV personalizado
- Dark / Light mode

React 19 | Vite 6 | Recharts | Electron-ready | Anthropic SDK

02

Chatbot Semántico AI

- Conectado a Anthropic API
- Modelo: claude-sonnet-4
- System prompt SRE-aware
- Contexto del dataset embebido
- Quick questions pre-definidas
- Historial de conversación
- Streaming SSE en NarrativePanel

Paso a Paso: Cómo lo construí

PASO 01 ~10 min

Leer el brief y explorar el CSV

Leí la prueba completa. Abrí el CSV y analicé 290 filas, intervalos de 10s, rango 06:11–07:00.

Decisión / Reto:
Acotar el análisis a una ventana horaria para hacer el dashboard

PASO 02 ~15 min

Diseñar la arquitectura con Claude

Le pedí a Claude que propusiera arquitectura React de dos paneles. Definimos componentes y modelos.

Decisión / Reto:
Separar lógica de datos (availabilityData.js) de los

PASO 03 ~20 min

Scaffolding y componentes base

Claude generó el proyecto Vite+React y la estructura. Yo revisé y ajusté la lógica con datos reales.

Decisión / Reto:
Hardcodear el dataset pre-procesado para garantizar la

PASO 04 ~25 min

Dashboard: KPIs y gráfico

Construí el gráfico con Recharts, las 5 KPI cards con skeleton loading y la línea SLO. 3 iteraciones.

Decisión / Reto:
Usar AreaChart con fill para visualizar el volumen acumulado de

PASO 05 ~20 min

Chatbot: integrar Claude API

Conecté la API directamente desde el browser. El reto mayor fue el error CORS al inicio.

Decisión / Reto:
Claude explicó el header anthropic-dangerous-direct-browser-access

PASO 06 ~15 min

Narrative Panel + SSE Streaming

Agregué el panel de narrativa con Haiku y streaming. El texto en tiempo real mejora la percepción.

Decisión / Reto:
Parsear el stream SSE línea a línea sin perder chunks parciales

PASO 07 ~15 min

SRE Framework: Golden Signals + SLO

Apliqué el Google SRE Book para los paneles inferiores. Detección de plateau con análisis de deltas.

Decisión / Reto:
Usar Golden Signals como marco conceptual demostraba conocimiento

PASO 08 ~10 min

Polish: SplashScreen, dark mode, CSV upload

SplashScreen animado con logo Rappi, toggle dark/light y cargador de CSV propio.

Decisión / Reto:
La UX de onboarding (splash + skeleton) hacía la app sentirse

Cómo trabajé con Claude como Copiloto

EL CICLO DE TRABAJO

1. Yo defino el qué → 2. Claude propone el cómo → 3. Yo reviso y ajusto → 4. Integro al proyecto

Prompts de Diseño

"Quiero un dashboard React para visualizar series temporales de disponibilidad de tiendas. ¿Cómo estructurarías los componentes separando datos de UI?"

Prompts de Implementación

"El NarrativePanel debe usar claude-haiku con streaming SSE. Necesito que el texto aparezca en pantalla mientras llega el stream. Implementa el lector de SSE."

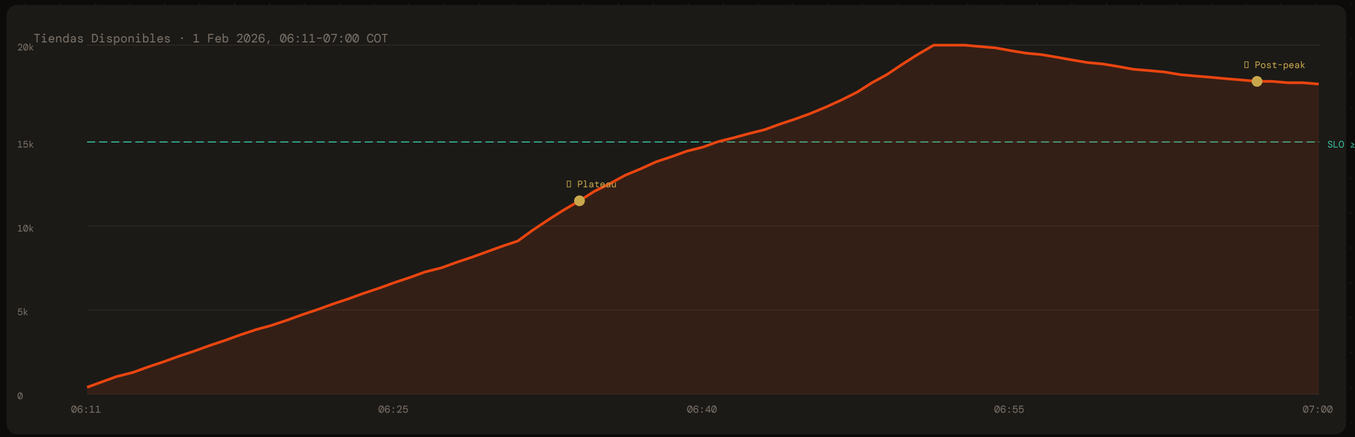
Prompts de Debug

"Tengo un error CORS al llamar a la Anthropic API desde el browser. ¿Qué header especial necesito para habilitar acceso directo desde el navegador?"

YO DECIDÍ	CLAUDE EJECUTÓ
Qué datos mostrar y con qué lógica	Estructura de componentes React
Qué modelo usar en cada componente	Boilerplate y configuración Vite
El system prompt del chatbot SRE	Implementación del loop SSE
Los criterios de detección de anomalías	Solución al error de CORS
El marco SRE (Golden Signals + SLO)	CSS, dark mode, animaciones
Cuándo el código era aceptable o no	Esta presentación completa

LECCIÓN: Usar AI estratégicamente no es copiar y pegar. Es saber qué preguntarle, evaluar su respuesta y asumir responsabilidad por las decisiones. El criterio de ingeniería sigue siendo mío.

Visualización de Disponibilidad



20,762
Pico de Tiendas
Traffic · Golden Signal

18,751
Tiendas al Cierre
-9.7% vs pico

92.4%
SLO Compliance
Error budget: 7.6%

2
Anomalías AI
1 warning · 1 info

19,476
Promedio Ventana
290 pts · intervalo 10s

Cómo usamos Claude & la Anthropic API

Chatbot SRE

claude-sonnet-4

- System prompt con rol SRE
- Contexto del dataset embebido
- Historial de conversación
- 800 tokens de respuesta
- Direct browser API access
- Quick questions preloaded

```
model: "claude-sonnet-4-20250514"
```

Narrative Panel

claude-haiku-4-5

- Auto-genera narrativa ejecutiva
- Streaming SSE en tiempo real
- 3 oraciones para Engineering Mgr
- Se actualiza al cambiar CSV
- max_tokens: 320
- Detección de plateau AI

```
model: "claude-haiku-4-5-20251001"
```

Anomaly Detection

Análisis estadístico + AI

- Detección de plateau 06:33–06:39
- Post-peak drop identification
- Context-aware via Claude API
- Severidad: warning / info
- Integrado con dataset CSV
- Tiempo real al cargar datos

```
avgDelta < 0.1 → plateau detected
```

¿Por qué estas decisiones?

Sonnet para razonamiento complejo · Haiku para velocidad y bajo costo · Ambos via Anthropic API directa desde el browser

Golden Signals & SLO Analysis

TRAFFIC

20,762

stores peak

Carga máxima del sistema en la ventana analizada.

ok

SATURATION

90%

del pico al cierre

Saturación aceptable. −9.7% post-pico es normal.

ok

ERRORS

N/A

sin datos de error

El CSV no contiene error rate. En prod: HTTP 5xx / failed probes.

info

LATENCY

10s

intervalo de sondeo

Monitoreo cada 10 segundos. Resolution: suficiente para análisis.

ok

SLO ANALYSIS

SLO TARGET	COMPLIANCE	ERROR BUDGET	PERIODO BAJO SLO
□ 15,000 tiendas disponibles	92.4% del tiempo en ventana	7.6% consumido (□ 3.2 min bajo SLO)	06:11–06:25 (ramp-up fase inicial)

Stack & Decisiones Técnicas

¿FUE ESTRATÉGICO O COPY/PASTE?

Scaffolding

Estructura inicial del proyecto y componentes React generados con Claude Cowork

System Prompts

Diseño iterativo del rol SRE en ChatBot y NarrativePanel

Debug

Resolución de errores CORS, SSE streaming, y ReportLab para esta presentación

UX Decisions

Paleta de colores, layout de KPIs y lógica de SplashScreen sugeridos por

React 19

UI Framework

Composición por componentes, Hooks para estado reactivo. Cada panel del

Vite 6

Build Tool + Dev Server

Hot Module Replacement instantáneo. Variables de entorno para la API key vía .env /

Recharts

Data Visualization

Librería declarativa basada en SVG. Permite AreaChart, ReferenceLine para

Anthropic API

AI Backend

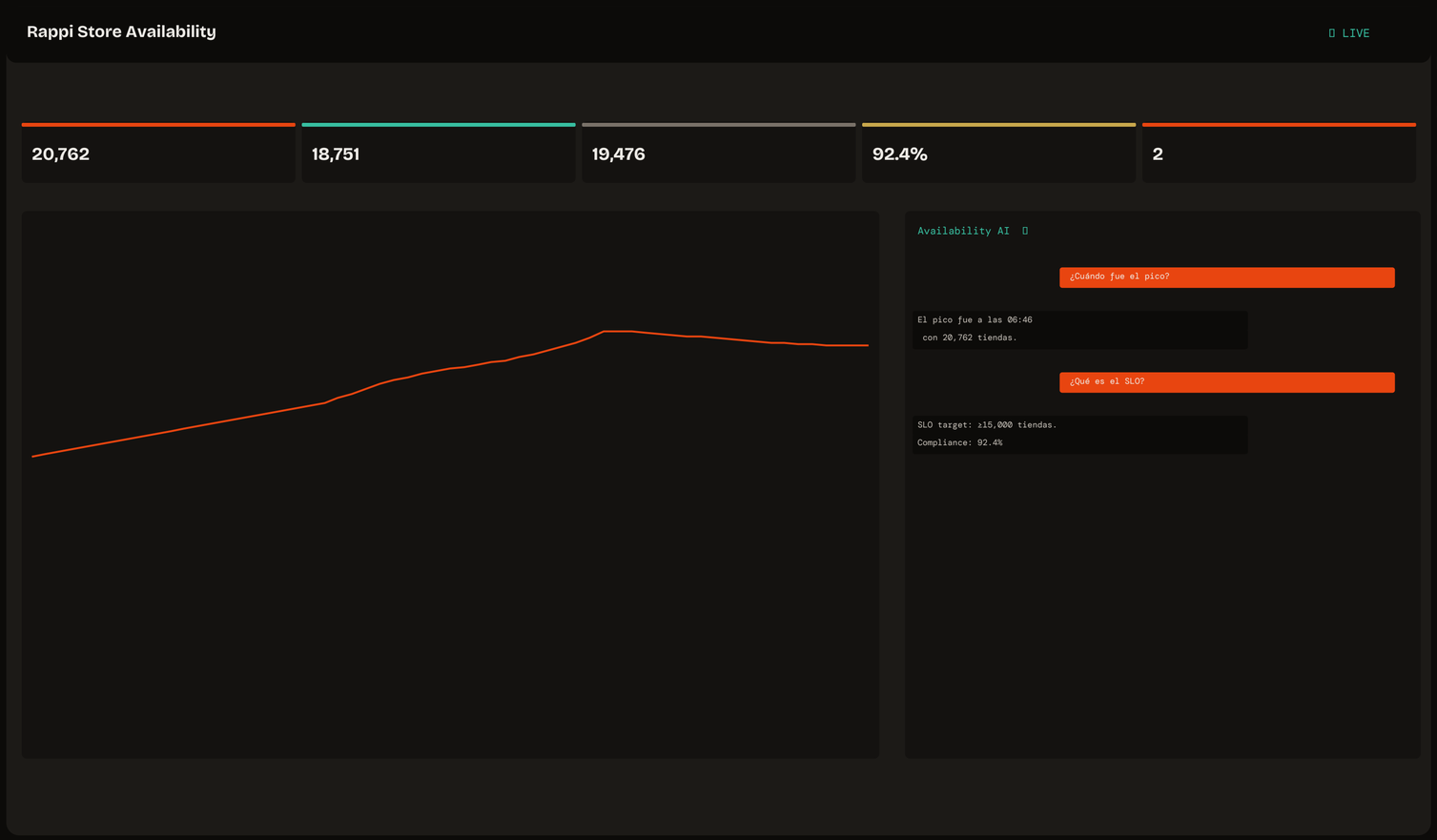
Direct browser access con anthropic-dangerous-direct-browser-access. SSE streaming para

Claude in Cowork

AI Development Tool

Usado para scaffolding, debugging de componentes React, diseño de system

Demo & Resultados



- App 100% funcional localmente
- Dashboard con datos reales del CSV
- Chatbot responde sobre SLOs y KPIs
- Narrativa AI en streaming (SSE)
- Dark/Light mode
- Carga de CSV personalizado
- SplashScreen animado con logo Rappi
- Golden Signals framework (Google SRE)
- 2 modelos Claude distintos por caso de uso

CÓMO EJECUTAR

```
$ npm install
$ echo VITE_ANTHROPIC_API_KEY=sk-... > .env
$ npm run dev  →  localhost:5173
```

Arquitectura del Código Fuente

ChatBot.jsx · Anthropic API Call — claude-sonnet-4

```
// llamada directa desde el browser
const response = await fetch(
  'https://api.anthropic.com/v1/messages',
  { headers: {
    'x-api-key': apiKey,
    'anthropic-dangerous-direct-browser-access': 'true',
  }, body: JSON.stringify({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 800, system: SYSTEM_PROMPT,
    messages: apiHistory,
  }) }
)
```

NarrativePanel.jsx · SSE Streaming — claude-haiku-4-5

```
// Leer stream SSE en tiempo real
const reader  = res.body.getReader()
const decoder = new TextDecoder()
while (true) {
  const { done, value } = await reader.read()
  if (done) break
  const chunk = decoder.decode(value, { stream: true })
  for (const line of chunk.split('\n')) {
    if (!line.startsWith('data: ')) continue
    const parsed = JSON.parse(line.slice(6))
    if (parsed.type === 'content_block_delta')
      setText(prev => prev + parsed.delta.text)
  }
}
```

App.jsx · Layout principal — React 19

```
return (
  <div>
    <SplashScreen onFinish={...} />
    <Header darkMode={darkMode} />
    <main className='layout-main'>
      <section className='grid-kpis'>
        <KpiCard label='Pico' value={stats.max} />
      </section>
      <AvailabilityChart /> <ChatBot />
      <NarrativePanel series={series} />
      <GoldenSignals /> <AnomalyPanel />
    </main>
  </div>
)
```

NarrativePanel.jsx · Detección de Plateau — Anomaly Detection

```
// Detección estadística de plateau
const deltas = sampled.slice(1).map(
  (d, i) => Math.abs(d.value - sampled[i].value)
)
const avgDelta = deltas.reduce((a,b) => a+b, 0)
               / (deltas.length || 1)
const flatCount = deltas.filter(
  d => d < avgDelta * 0.1
).length
// flatCount > 6 □ plateau detectado
const plateauNote = flatCount > 6
  ? `Plateau: ${flatCount} segs`
  : ''
```

src/ App.jsx ChatBot.jsx NarrativePanel.jsx AvailabilityChart.jsx GoldenSignals.jsx AnomalyPanel.jsx SloSummary.jsx KpiCard.jsx availabilityData.js

Referencias & Bibliografías

01 **Google SRE Book — Site Reliability Engineering**
Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.) · 2016
O'Reilly Media · sre.google/sre-book/table-of-contents/
Base conceptual de Golden Signals (Traffic, Saturation, Errors, Latency) y SLO/SLI framework

02 **Anthropic Claude API Documentation**
Anthropic, PBC · 2025
docs.anthropic.com/en/api/getting-started
Referencia de la Messages API, streaming SSE, modelos claude-sonnet-4 y claude-haiku-4-5

03 **React Documentation — React 19**
Meta Open Source · 2024
react.dev/reference/react
Hooks (useState, useEffect, useRef, useMemo), componentes funcionales y composición de UI

04 **Recharts — A composable charting library for React**
recharts.org · 2024
recharts.org/api
AreaChart, ReferenceLine, Tooltip personalizable para la visualización de series temporales

05 **Vite — Next Generation Frontend Tooling**
Evan You & Vite contributors · 2024
vitejs.dev/guide/
Build tool, HMR, variables de entorno (import.meta.env) para API key management

06 **Server-Sent Events (SSE) — MDN Web Docs**
Mozilla Developer Network · 2024
developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
Protocolo de streaming usado en NarrativePanel para recibir respuestas de Claude en tiempo real